Практическая работа №2: CRUD-операции с различными базами данных

Цель работы

Освоить выполнение базовых операций создания (Create), чтения (Read), обновления (Update) и удаления (Delete) данных в пяти различных системах управления базами данных: **Redis**, **MongoDB**, **Neo4j**, **Elasticsearch** и **PostgreSQL**. Изучить специфику работы с каждой СУБД, понять различия в моделях данных и синтаксисе запросов.

Требования

- Выполненная Практическая работа №1 (все пять БД развёрнуты в Docker-контейнерах и доступны).
- Установленные клиенты для подключения к каждой БД (либо использование клиентов внутри контейнеров).

Предметная область

Во всех примерах используется единая предметная область — **интернет-магазин электроники**. Это позволяет сравнить, как одни и те же данные представляются в разных моделях хранения.

Теоретическая часть

Что такое CRUD

CRUD — акроним, описывающий четыре базовые операции над данными:

Операция	Описание	SQL-аналог
Create	Создание новых записей	`INSERT`
Read	Чтение (выборка) данных	`SELECT`

Update	Изменение существующих данных	`UPDATE`
Delete	Удаление данных	`DELETE`

Каждая СУБД реализует эти операции по-своему в зависимости от модели данных.

Модели данных в рассматриваемых СУБД

СУБД	Модель данных	Единица хранения
Redis	Ключ-значение	Ключ → значение (строка, хэш, список, множество, sorted set)
MongoDB	Документная	Документ (JSON/BSON) в коллекции
Neo4j	Графовая	Узел (node) и связь (relationship)
Elasticsearch	Поисковый движок	Документ (JSON) в индексе
PostgreSQL	Реляционная	Строка (row) в таблице

Подключение к СУБД

Предполагается, что контейнеры запущены из Практической работы №1. Ниже приведены команды подключения к клиентам внутри контейнеров:

```
# Redis
docker exec -it redis redis-cli

# MongoDB
docker exec -it mongodb mongosh

# Neo4j — веб-интерфейс http://localhost:7474 или CLI:
docker exec -it neo4j cypher-shell -u neo4j -p password

# Elasticsearch — HTTP API через curl:
curl -X GET "http://localhost:9200"

# PostgreSQL
docker exec -it postgres psql -U postgres
```

> ***Примечание.*** *Имена контейнеров (`redis`, `mongodb`, `neo4j`, `postgres`) и пароли могут отличаться в зависимости от вашего `docker-compose.yml`. Подставьте свои значения.*

Часть 1. Redis

1.1. Теория

Redis (Remote Dictionary Server) — резидентная (in-memory) СУБД типа «ключ-значение». Особенности:

- Все данные хранятся в оперативной памяти, что обеспечивает минимальную латентность.
- Поддерживает несколько типов данных: строки (strings), хэши (hashes), списки (lists), множества (sets), упорядоченные множества (sorted sets).
- Нет понятия «таблица» или «коллекция» — есть пространство ключей (keyspace).
- Нет языка запросов — используются команды протокола Redis.

Основные типы данных Redis:

Тип	Описание	Пример использования
String	Простое значение (текст, число, бинарные данные)	Кэширование, счётчики
Hash	Набор пар «поле-значение» (аналог объекта)	Хранение сущностей (пользователь, товар)
List	Упорядоченный список строк	Очереди сообщений, история
Set	Неупорядоченное множество уникальных строк	Теги, уникальные посетители
Sorted Set	Множество с весом (score) у каждого элемента	Рейтинги, лидерборды

1.2. Подключение

```
docker exec -it redis redis-cli
```

После подключения вы увидите приглашение `127.0.0.1:6379>`.

Проверка связи:

```
127.0.0.1:6379> PING
PONG
```

1.3. Операции со строками (Strings)

Создание (Create)

Команда `SET` устанавливает значение по ключу.

```
SET product:1:name "Смартфон Samsung Galaxy S24"
```

Синтаксис: `SET ключ значение`

Ожидаемый результат:

```
OK
```

Можно задать несколько ключей одной командой с помощью `MSET`:

```
MSET product:1:price "79990" product:1:category "Смартфоны"
product:1:stock "150"
```

Ожидаемый результат:

```
OK
```

Установка ключа с временем жизни (TTL) — полезно для кэширования:

```
SET session:user:42 "active" EX 3600
```

Пояснение: `EX 3600` — ключ будет автоматически удалён через 3600 секунд (1 час).

Чтение (Read)

Команда `GET` возвращает значение по ключу:

```
GET product:1:name
```

Ожидаемый результат:

```
"Смартфон Samsung Galaxy S24"
```

Получение нескольких значений одной командой:

```
MGET product:1:name product:1:price product:1:category
```

Ожидаемый результат:

```
1) "Смартфон Samsung Galaxy S24"
2) "79990"
3) "Смартфоны"
```

Проверка существования ключа:

```
EXISTS product:1:name
```

Ожидаемый результат: `(integer) 1` (1 — ключ существует, 0 — не существует).

Проверка оставшегося времени жизни:

```
TTL session:user:42
```

Ожидаемый результат: число секунд до удаления (например, `(integer) 3595`). Значение `-1` означает, что TTL не задан; `-2` — ключ не существует.

Обновление (Update)

Перезапись значения — та же команда `SET`:

```
SET product:1:price "74990"
```

Инкремент числового значения (атомарная операция):

```
INCR product:1:stock
```

Ожидаемый результат: `(integer) 151` — значение увеличилось на 1.

Декремент:

```
DECRBY product:1:stock 10
```

Ожидаемый результат: `(integer) 141` — значение уменьшилось на 10.

Добавление текста к существующему значению:

```
APPEND product:1:name " (Ultra)"
```

Ожидаемый результат: `(integer) 41` — возвращается длина итоговой строки.

```
GET product:1:name
```

Ожидаемый результат:

```
"Смартфон Samsung Galaxy S24 (Ultra)"
```

Удаление (Delete)

Удаление одного или нескольких ключей:

```
DEL product:1:category
```

Ожидаемый результат: `(integer) 1` — количество удалённых ключей.

Удаление нескольких ключей:

```
DEL product:1:name product:1:price product:1:stock
```

Ожидаемый результат: `(integer) 3` .

1.4. Операции с хэшами (Hashes)

Хэш — это набор пар «поле-значение», привязанный к одному ключу. Идеально подходит для хранения объектов.

Создание

```
HSET product:2 name "Ноутбук ASUS VivoBook" price 64990 category
"Ноутбуки" stock 75 brand "ASUS"
```

Синтаксис: `HSET ключ поле1 значение1 поле2 значение2 ...`

Ожидаемый результат: `(integer) 5` — количество добавленных полей.

Чтение

Получить одно поле:

```
HGET product:2 name
```

Ожидаемый результат:

```
"Ноутбук ASUS VivoBook"
```

Получить несколько полей:

```
HMGET product:2 name price stock
```

Ожидаемый результат:

```
1) "Ноутбук ASUS VivoBook"
2) "64990"
3) "75"
```

Получить все поля и значения:

```
HGETALL product:2
```

Ожидаемый результат:

```
 1) "name"
 2) "Ноутбук ASUS VivoBook"
 3) "price"
 4) "64990"
 5) "category"
 6) "Ноутбуки"
 7) "stock"
 8) "75"
 9) "brand"
10) "ASUS"
```

Получить только имена полей или только значения:

```
HKEYS product:2
HVALS product:2
```

Проверить существование поля в хэше:

```
HEXISTS product:2 brand
```

Ожидаемый результат: `(integer) 1`.

Обновление

Изменить значение поля (используется та же команда `HSET`):

```
HSET product:2 price 59990
```

Ожидаемый результат: `(integer) 0` — 0 означает, что поле уже существовало и было обновлено (а не создано заново).

Атомарный инкремент числового поля:

```
HINCRBY product:2 stock -5
```

Ожидаемый результат: `(integer) 70` — остаток на складе уменьшился на 5.

Добавить новое поле к существующему хэшу:

```
HSET product:2 color "Серебристый" weight "1.7 кг"
```

Ожидаемый результат: `(integer) 2` — добавлено 2 новых поля.

Удаление

Удалить отдельное поле из хэша:

```
HDEL product:2 color
```

Ожидаемый результат: `(integer) 1`.

Удалить весь хэш (как обычный ключ):

```
DEL product:2
```

1.5. Операции со списками (Lists)

Список — упорядоченная последовательность строк. Поддерживает добавление с обоих концов.

Создание

Добавление элементов в начало списка (слева):

```
LPUSH order:1001:items "Смартфон Samsung Galaxy S24" "Чехол для Samsung" "Защитное стекло"
```

Синтаксис: `LPUSH ключ значение1 значение2 ...` — элементы вставляются слева (в начало). Последний указанный элемент окажется первым в списке.

Ожидаемый результат: `(integer) 3` — длина списка после вставки.

Добавление элементов в конец списка (справа):

```
RPUSH order:1001:items "Наушники TWS"
```

Ожидаемый результат: `(integer) 4` .

Чтение

Получить все элементы списка:

```
LRANGE order:1001:items 0 -1
```

Синтаксис: `LRANGE ключ начало конец` — индексы начинаются с 0, `-1` означает последний элемент.

Ожидаемый результат:

```
1) "Защитное стекло"
2) "Чехол для Samsung"
3) "Смартфон Samsung Galaxy S24"
4) "Наушники TWS"
```

Получить элемент по индексу:

```
LINDEX order:1001:items 0
```

Ожидаемый результат: `"Защитное стекло"`.

Узнать длину списка:

```
LLEN order:1001:items
```

Ожидаемый результат: `(integer) 4`.

Обновление

Заменить элемент по индексу:

```
LSET order:1001:items 2 "Смартфон Samsung Galaxy S24 Ultra"
```

Ожидаемый результат: `OK`.

Вставить элемент перед или после указанного значения:

```
LINSERT order:1001:items BEFORE "Наушники TWS" "Кабель USB-C"
```

Ожидаемый результат: `(integer) 5` — новая длина списка.

Удаление

Удалить и вернуть элемент из начала списка:

```
LPOP order:1001:items
```

Ожидаемый результат: `"Защитное стекло"`.

Удалить и вернуть элемент из конца списка:

```
RPOP order:1001:items
```

Ожидаемый результат: `"Наушники TWS"`.

Удалить конкретный элемент по значению:

```
LREM order:1001:items 1 "Кабель USB-C"
```

Синтаксис: `LREM ключ количество значение` — `1` означает удалить первое вхождение.

Удалить весь список:

```
DEL order:1001:items
```

1.6. Операции с множествами (Sets)

Множество — неупорядоченная коллекция уникальных строк.

Создание

```
SADD product:tags:smartphone "Android" "5G" "NFC" "AMOLED"
"128GB"
SADD product:tags:laptop "Windows" "SSD" "FullHD" "USB-C"
```

Ожидаемый результат: `(integer) 5` и `(integer) 4` соответственно.

Чтение

Получить все элементы множества:

```
SMEMBERS product:tags:smartphone
```

Ожидаемый результат:

```
1) "AMOLED"
2) "5G"
3) "NFC"
4) "Android"
5) "128GB"
```

> ***Важно:*** *порядок элементов не гарантирован.*

Проверить принадлежность элемента множеству:

```
SISMEMBER product:tags:smartphone "NFC"
```

Ожидаемый результат: `(integer) 1` (принадлежит).

Количество элементов в множестве:

```
SCARD product:tags:smartphone
```

Ожидаемый результат: `(integer) 5` .

Операции над множествами (пересечение, объединение, разность):

```
SINTER product:tags:smartphone product:tags:laptop
SUNION product:tags:smartphone product:tags:laptop
SDIFF product:tags:smartphone product:tags:laptop
```

Обновление

Добавить новый элемент:

```
SADD product:tags:smartphone "WiFi 6"
```

Ожидаемый результат: `(integer) 1`.

Удаление

Удалить конкретный элемент:

```
SREM product:tags:smartphone "128GB"
```

Ожидаемый результат: `(integer) 1`.

Удалить всё множество:

```
DEL product:tags:smartphone product:tags:laptop
```

1.7. Операции с упорядоченными множествами (Sorted Sets)

Каждый элемент имеет числовой вес (score), по которому выполняется сортировка.

Создание

```
ZADD product:rating 4.8 "Samsung Galaxy S24" 4.6 "iPhone 15" 4.9
"Pixel 8 Pro" 4.3 "Xiaomi 14" 4.7 "OnePlus 12"
```

Синтаксис: `ZADD ключ score1 member1 score2 member2 ...`

Ожидаемый результат: `(integer) 5`.

Чтение

Получить элементы по возрастанию рейтинга:

```
ZRANGE product:rating 0 -1 WITHSCORES
```

Ожидаемый результат:

```
 1) "Xiaomi 14"
 2) "4.3"
 3) "iPhone 15"
 4) "4.6"
 5) "OnePlus 12"
 6) "4.7"
 7) "Samsung Galaxy S24"
 8) "4.8"
 9) "Pixel 8 Pro"
10) "4.9"
```

Получить топ-3 по убыванию:

```
ZREVRANGE product:rating 0 2 WITHSCORES
```

Ожидаемый результат:

```
1) "Pixel 8 Pro"
2) "4.9"
3) "Samsung Galaxy S24"
4) "4.8"
5) "OnePlus 12"
6) "4.7"
```

Узнать ранг (позицию) элемента:

```
ZRANK product:rating "Samsung Galaxy S24"
```

Ожидаемый результат: `(integer) 3` (нумерация с 0, по возрастанию).

Получить элементы в диапазоне score:

```
ZRANGEBYSCORE product:rating 4.5 5.0 WITHSCORES
```

Ожидаемый результат: все товары с рейтингом от 4.5 до 5.0.

Обновление

Изменить score элемента:

```
ZADD product:rating 4.5 "Xiaomi 14"
```

Ожидаемый результат: `(integer) 0` — элемент уже существует, score обновлён.

Инкрементировать score:

```
ZINCRBY product:rating 0.1 "iPhone 15"
```

Ожидаемый результат: `"4.7"` — новое значение score.

Удаление

Удалить элемент из sorted set:

```
ZREM product:rating "Xiaomi 14"
```

Ожидаемый результат: `(integer) 1` .

Удалить элементы по диапазону score:

```
ZREMRANGEBYSCORE product:rating 0 4.5
```

Удалить всё хранилище:

```
DEL product:rating
```

1.8. Удаление всех данных в Redis

Удалить все ключи в текущей базе данных:

```
FLUSHDB
```

Удалить все ключи во всех базах данных:

```
FLUSHALL
```

> ***Внимание!*** *Эти команды необратимы. Используйте с осторожностью.*

Часть 2. MongoDB

2.1. Теория

MongoDB — документоориентированная СУБД. Основные понятия:

Понятие	Описание	Аналог в SQL
Database	База данных	Database
Collection	Коллекция документов	Table
Document	Документ (JSON/BSON)	Row
Field	Поле документа	Column
`_id`	Уникальный идентификатор документа	Primary Key

Особенности:

- Документы хранятся в формате BSON (Binary JSON).

- Схема гибкая — документы в одной коллекции могут иметь разный набор полей.
- Поддерживает вложенные документы и массивы.
- Мощный язык запросов и фреймворк агрегации.

2.2. Подключение

```
docker exec -it mongodb mongosh
```

Выбор (или создание) базы данных:

```
use electronics_shop
```

Пояснение: если база данных не существует, MongoDB создаст её автоматически при первой вставке данных.

2.3. Создание структуры данных и добавление документов (Create)

Вставка одного документа

```
db.products.insertOne({
  name: "Смартфон Samsung Galaxy S24",
  brand: "Samsung",
  price: 79990,
  category: "Смартфоны",
  specs: {
    display: "6.2 дюйма, AMOLED",
    processor: "Exynos 2400",
    ram: 8,
    storage: 128,
    battery: 4000
  },
  tags: ["Android", "5G", "NFC", "AMOLED"],
  inStock: true,
  reviews: [
    { user: "Иван", rating: 5, text: "Отличный смартфон!" },
    { user: "Мария", rating: 4, text: "Хорошая камера, но
```

```
дорого." }
  ],
  createdAt: new Date()
})
```

Пояснение:

- `db.products` — обращение к коллекции `products`. Если коллекция не существует, она создаётся автоматически.
- `specs` — вложенный документ (embedded document), содержащий технические характеристики.
- `tags` — массив строк.
- `reviews` — массив вложенных документов.
- `new Date()` — текущая дата и время.

Ожидаемый результат:

```
{
  "acknowledged": true,
  "insertedId": ObjectId("...")
}
```

Вставка нескольких документов

```
db.products.insertMany([
  {
    name: "Ноутбук ASUS VivoBook 15",
    brand: "ASUS",
    price: 64990,
    category: "Ноутбуки",
    specs: {
      display: "15.6 дюймов, IPS, FullHD",
      processor: "Intel Core i5-1235U",
      ram: 16,
      storage: 512,
      gpu: "Intel Iris Xe"
    },
    tags: ["Windows", "SSD", "FullHD", "USB-C"],
    inStock: true,
```

```
    reviews: [
      { user: "Алексей", rating: 4, text: "Хороший ноутбук для
учёбы." }
    ],
    createdAt: new Date()
  },
  {
    name: "Наушники Sony WH-1000XM5",
    brand: "Sony",
    price: 29990,
    category: "Аудио",
    specs: {
      type: "Накладные",
      connection: "Bluetooth 5.2",
      noiseCancellation: true,
      batteryLife: 30
    },
    tags: ["Bluetooth", "ANC", "Hi-Res"],
    inStock: true,
    reviews: [],
    createdAt: new Date()
  },
  {
    name: "Планшет iPad Air",
    brand: "Apple",
    price: 59990,
    category: "Планшеты",
    specs: {
      display: "10.9 дюймов, Liquid Retina",
      processor: "Apple M1",
      ram: 8,
      storage: 64
    },
    tags: ["iPadOS", "M1", "USB-C"],
    inStock: false,
    reviews: [
      { user: "Ольга", rating: 5, text: "Лучший планшет!" },
      { user: "Дмитрий", rating: 5, text: "Идеален для
рисования." }
    ],
    createdAt: new Date()
  },
  {
    name: "Телевизор LG OLED55C3",
    brand: "LG",
    price: 119990,
```

```
    category: "Телевизоры",
    specs: {
      display: "55 дюймов, OLED, 4K",
      smartTV: true,
      hdr: "Dolby Vision, HDR10",
      refreshRate: 120
    },
    tags: ["OLED", "4K", "SmartTV", "HDR"],
    inStock: true,
    reviews: [
      { user: "Сергей", rating: 5, text: "Потрясающая картинка!"
}
    ],
    createdAt: new Date()
  }
])
```

Ожидаемый результат:

```
{
  "acknowledged": true,
  "insertedIds": {
    "0": ObjectId("..."),
    "1": ObjectId("..."),
    "2": ObjectId("..."),
    "3": ObjectId("...")
  }
}
```

2.4. Чтение данных (Read)

Поиск всех документов

```
db.products.find()
```

Для более читаемого вывода:

```
db.products.find().pretty()
```

Поиск по конкретному полю

```
db.products.find({ brand: "Samsung" })
```

Ожидаемый результат: все документы, где `brand` равен `"Samsung"`.

Поиск по вложенному полю

Используется точечная нотация (dot notation):

```
db.products.find({ "specs.ram": { $gte: 8 } })
```

Пояснение: `$gte` — оператор «больше или равно» (greater than or equal). Найдёт все товары с RAM >= 8 ГБ.

Поиск по элементу массива

```
db.products.find({ tags: "5G" })
```

Пояснение: MongoDB автоматически ищет значение среди элементов массива.

Поиск по вложенному документу в массиве

```
db.products.find({ "reviews.rating": 5 })
```

Ожидаемый результат: все товары, у которых есть хотя бы один отзыв с рейтингом 5.

Операторы сравнения

```
// Цена от 30000 до 80000
db.products.find({ price: { $gte: 30000, $lte: 80000 } })

// Бренд — один из списка
db.products.find({ brand: { $in: ["Samsung", "Apple", "Sony"] }
})

// Товары в наличии
db.products.find({ inStock: true })
```

Основные операторы:

Оператор	Описание
`$eq`	Равно
`$ne`	Не равно
`$gt`	Больше
`$gte`	Больше или равно
`$lt`	Меньше
`$lte`	Меньше или равно
`$in`	Входит в массив значений
`$nin`	Не входит в массив значений
`$exists`	Поле существует
`$regex`	Регулярное выражение

Проекция (выбор полей)

```
db.products.find(
  { category: "Смартфоны" },
  { name: 1, price: 1, _id: 0 }
)
```

Пояснение: второй аргумент — проекция. `1` — включить поле, `0` — исключить. `_id` включается по умолчанию, поэтому его нужно явно исключать.

Ожидаемый результат:

```
{ "name": "Смартфон Samsung Galaxy S24", "price": 79990 }
```

Сортировка, лимит, пропуск

```
// Сортировка по цене (по убыванию), вывод первых 3
db.products.find().sort({ price: -1 }).limit(3)

// Пропустить первые 2 и вернуть следующие 2 (пагинация)
db.products.find().skip(2).limit(2)
```

Подсчёт документов

```
db.products.countDocuments({ inStock: true })
```

Ожидаемый результат: `4` (или другое число в зависимости от данных).

Поиск одного документа

```
db.products.findOne({ brand: "Sony" })
```

2.5. Обновление данных (Update)

Обновление одного документа

```
db.products.updateOne(
  { name: "Смартфон Samsung Galaxy S24" },
```

```
  { $set: { price: 74990, "specs.ram": 12 } }
)
```

Пояснение:

- Первый аргумент — фильтр (какой документ обновить).
- `$set` — оператор, устанавливающий новое значение полей. Остальные поля не затрагиваются.

Ожидаемый результат:

```
{
  "acknowledged": true,
  "matchedCount": 1,
  "modifiedCount": 1
}
```

Добавление элемента в массив

```
db.products.updateOne(
  { name: "Наушники Sony WH-1000XM5" },
  {
    $push: {
      reviews: {
        user: "Елена",
        rating: 5,
        text: "Лучшее шумоподавление на рынке!"
      }
    }
  }
)
```

Пояснение: `$push` добавляет элемент в конец массива.

Добавление нескольких элементов в массив

```
db.products.updateOne(
  { name: "Наушники Sony WH-1000XM5" },
```

```
  {
    $push: {
      tags: { $each: ["LDAC", "Multipoint"] }
    }
  }
)
```

Удаление элемента из массива

```
db.products.updateOne(
  { name: "Смартфон Samsung Galaxy S24" },
  {
    $pull: { tags: "128GB" }
  }
)
```

Пояснение: `$pull` удаляет все вхождения указанного значения из массива.

Инкремент числового поля

```
db.products.updateOne(
  { name: "Планшет iPad Air" },
  { $inc: { price: -5000 } }
)
```

Пояснение: `$inc` увеличивает значение поля. Отрицательное число — уменьшает. Цена уменьшится на 5000.

Обновление нескольких документов

```
db.products.updateMany(
  { inStock: true },
  { $set: { updatedAt: new Date() } }
)
```

Ожидаемый результат:

```
{
  "acknowledged": true,
  "matchedCount": 4,
  "modifiedCount": 4
}
```

Добавление нового поля (upsert)

```
db.products.updateOne(
  { name: "Смарт-часы Apple Watch SE" },
  {
    $set: {
      brand: "Apple",
      price: 24990,
      category: "Носимая электроника",
      specs: { display: "40 мм, OLED", waterproof: true },
      tags: ["watchOS", "GPS"],
      inStock: true,
      createdAt: new Date()
    }
  },
  { upsert: true }
)
```

Пояснение: `upsert: true` — если документ с таким фильтром не найден, он будет создан. Если найден — обновлён.

2.6. Агрегация

Фреймворк агрегации позволяет выполнять сложные аналитические запросы.

Средняя цена по категориям

```
db.products.aggregate([
  {
    $group: {
      _id: "$category",
      avgPrice: { $avg: "$price" },
      count: { $sum: 1 },
```

```
      maxPrice: { $max: "$price" },
      minPrice: { $min: "$price" }
    }
  },
  { $sort: { avgPrice: -1 } }
])
```

Пояснение:

- `$group` — группировка. `_id: "$category"` — группировать по полю `category`.
- `$avg`, `$sum`, `$max`, `$min` — агрегатные функции.
- `$sort` — сортировка результатов.

Ожидаемый результат:

```
[
  { "_id": "Телевизоры", "avgPrice": 119990, "count": 1,
"maxPrice": 119990, "minPrice": 119990 },
  { "_id": "Смартфоны", "avgPrice": 74990, "count": 1,
"maxPrice": 74990, "minPrice": 74990 },
  ...
]
```

Развернуть массив и посчитать популярность тегов

```
db.products.aggregate([
  { $unwind: "$tags" },
  {
    $group: {
      _id: "$tags",
      count: { $sum: 1 }
    }
  },
  { $sort: { count: -1 } },
  { $limit: 10 }
])
```

Пояснение: `$unwind` разворачивает массив — создаёт отдельный документ для каждого элемента массива `tags`.

2.7. Удаление данных (Delete)

Удаление одного документа

```
db.products.deleteOne({ name: "Смарт-часы Apple Watch SE" })
```

Ожидаемый результат:

```
{ "acknowledged": true, "deletedCount": 1 }
```

Удаление нескольких документов

```
db.products.deleteMany({ inStock: false })
```

Удаление всех документов в коллекции (без удаления коллекции)

```
db.products.deleteMany({})
```

Удаление коллекции

```
db.products.drop()
```

Ожидаемый результат: `true`.

Удаление базы данных

```
db.dropDatabase()
```

Ожидаемый результат:

```
{ "ok": 1, "dropped": "electronics_shop" }
```

Часть 3. Neo4j

3.1. Теория

Neo4j — графовая СУБД. Основные понятия:

Понятие	Описание	Обозначение в Cypher
Node (Узел)	Сущность (объект)	`(n)`
Label (Метка)	Тип/категория узла	`(n:Person)`
Relationship (Связь)	Направленная связь между узлами	`-[r:KNOWS]->`
Property (Свойство)	Атрибут узла или связи	`{name: "Иван"}`

Cypher — декларативный язык запросов для Neo4j (аналог SQL для графовых БД).

Основные ключевые слова Cypher:

Команда	Описание
`CREATE`	Создать узел или связь
`MATCH`	Найти по шаблону
`RETURN`	Вернуть результат
`SET`	Установить свойства

`DELETE`	Удалить узел/связь
`DETACH DELETE`	Удалить узел вместе со всеми связями
`MERGE`	Найти или создать (аналог upsert)
`WHERE`	Условие фильтрации
`WITH`	Промежуточный результат (аналог подзапроса)
`ORDER BY`	Сортировка

3.2. Подключение

Через веб-интерфейс: откройте в браузере `http://localhost:7474`.

Через CLI:

```
docker exec -it neo4j cypher-shell -u neo4j -p password
```

3.3. Создание структуры данных (Create)

Создание узлов

```
// Создание категорий
CREATE (c1:Category {name: "Смартфоны", description: "Мобильные телефоны и смартфоны"})
CREATE (c2:Category {name: "Ноутбуки", description: "Портативные компьютеры"})
CREATE (c3:Category {name: "Аудио", description: "Наушники и аудиотехника"})
RETURN c1, c2, c3;
```

Пояснение:

- `CREATE` — команда создания.
- `(c1:Category {...})` — создаётся узел с меткой `Category` и свойствами внутри `{...}`. Переменная `c1` используется для ссылки на узел в рамках запроса.

- `RETURN` — возвращает созданные узлы.

Ожидаемый результат: три строки с созданными узлами.

```
// Создание брендов
CREATE (b1:Brand {name: "Samsung", country: "Южная Корея",
founded: 1938})
CREATE (b2:Brand {name: "ASUS", country: "Тайвань", founded:
1989})
CREATE (b3:Brand {name: "Sony", country: "Япония", founded:
1946})
RETURN b1, b2, b3;
```

```
// Создание товаров
CREATE (p1:Product {name: "Galaxy S24", price: 79990, inStock:
true, rating: 4.8})
CREATE (p2:Product {name: "VivoBook 15", price: 64990, inStock:
true, rating: 4.6})
CREATE (p3:Product {name: "WH-1000XM5", price: 29990, inStock:
true, rating: 4.9})
RETURN p1, p2, p3;
```

```
// Создание покупателей
CREATE (u1:Customer {name: "Иван Петров", email:
"ivan@example.com", age: 28})
CREATE (u2:Customer {name: "Мария Сидорова", email:
"maria@example.com", age: 34})
CREATE (u3:Customer {name: "Алексей Козлов", email:
"alexey@example.com", age: 22})
RETURN u1, u2, u3;
```

Создание связей (Relationships)

```
// Связь товаров с категориями
MATCH (p:Product {name: "Galaxy S24"}), (c:Category {name:
"Смартфоны"})
```

```
CREATE (p)-[:BELONGS_TO]->(c)
RETURN p, c;
```

Пояснение:

- `MATCH` находит существующие узлы.
- `CREATE (p)-[:BELONGS_TO]->(c)` — создаёт направленную связь типа `BELONGS_TO` от товара к категории.

```
MATCH (p:Product {name: "VivoBook 15"}), (c:Category {name:
"Ноутбуки"})
CREATE (p)-[:BELONGS_TO]->(c);

MATCH (p:Product {name: "WH-1000XM5"}), (c:Category {name:
"Аудио"})
CREATE (p)-[:BELONGS_TO]->(c);
```

```
// Связь товаров с брендами
MATCH (p:Product {name: "Galaxy S24"}), (b:Brand {name:
"Samsung"})
CREATE (p)-[:MANUFACTURED_BY]->(b);

MATCH (p:Product {name: "VivoBook 15"}), (b:Brand {name:
"ASUS"})
CREATE (p)-[:MANUFACTURED_BY]->(b);

MATCH (p:Product {name: "WH-1000XM5"}), (b:Brand {name: "Sony"})
CREATE (p)-[:MANUFACTURED_BY]->(b);
```

```
// Покупки (связи со свойствами)
MATCH (u:Customer {name: "Иван Петров"}), (p:Product {name:
"Galaxy S24"})
CREATE (u)-[:PURCHASED {date: date("2025-01-15"), quantity: 1,
totalPrice: 79990}]->(p);

MATCH (u:Customer {name: "Иван Петров"}), (p:Product {name: "WH-
1000XM5"})
CREATE (u)-[:PURCHASED {date: date("2025-01-15"), quantity: 1,
```

```
totalPrice: 29990}]->(p);

MATCH (u:Customer {name: "Мария Сидорова"}), (p:Product {name:
"VivoBook 15"})
CREATE (u)-[:PURCHASED {date: date("2025-02-20"), quantity: 1,
totalPrice: 64990}]->(p);

MATCH (u:Customer {name: "Алексей Козлов"}), (p:Product {name:
"Galaxy S24"})
CREATE (u)-[:PURCHASED {date: date("2025-03-01"), quantity: 2,
totalPrice: 159980}]->(p);
```

Пояснение: связи тоже могут иметь свойства — дата покупки, количество, сумма.

```
// Отзывы
MATCH (u:Customer {name: "Иван Петров"}), (p:Product {name:
"Galaxy S24"})
CREATE (u)-[:REVIEWED {rating: 5, text: "Отличный смартфон!",
date: date("2025-01-20")}]->(p);

MATCH (u:Customer {name: "Мария Сидорова"}), (p:Product {name:
"VivoBook 15"})
CREATE (u)-[:REVIEWED {rating: 4, text: "Хороший ноутбук для
работы.", date: date("2025-03-01")}]->(p);
```

3.4. Чтение данных (Read)

Получить все узлы определённого типа

```
MATCH (p:Product)
RETURN p.name, p.price, p.rating
ORDER BY p.price DESC;
```

Ожидаемый результат:

p.name	p.price	p.rating
"Galaxy S24"	79990	4.8

"VivoBook 15"	64990	4.6
"WH-1000XM5"	29990	4.9

Найти товары конкретного бренда

```
MATCH (p:Product)-[:MANUFACTURED_BY]->(b:Brand {name:
"Samsung"})
RETURN p.name, p.price;
```

Пояснение: шаблон `(p)-[:MANUFACTURED_BY]->(b)` ищет узлы, связанные отношением `MANUFACTURED_BY`.

Найти все покупки пользователя

```
MATCH (u:Customer {name: "Иван Петров"})-[r:PURCHASED]->
(p:Product)
RETURN p.name, r.date, r.quantity, r.totalPrice;
```

Найти покупателей конкретного товара

```
MATCH (u:Customer)-[r:PURCHASED]->(p:Product {name: "Galaxy
S24"})
RETURN u.name, u.email, r.date, r.quantity;
```

Сложный запрос: товары, которые покупали вместе (рекомендации)

```
MATCH (u:Customer)-[:PURCHASED]->(p1:Product {name: "Galaxy
S24"}),
      (u)-[:PURCHASED]->(p2:Product)
WHERE p2.name <> "Galaxy S24"
RETURN DISTINCT p2.name AS recommendation, COUNT(u) AS buyers
ORDER BY buyers DESC;
```

Пояснение: ищем пользователей, которые купили Galaxy S24, и смотрим, что ещё они покупали.

Получить средний рейтинг отзывов товара

```
MATCH (u:Customer)-[r:REVIEWED]->(p:Product {name: "Galaxy
S24"})
RETURN p.name, AVG(r.rating) AS avgRating, COUNT(r) AS
reviewCount;
```

Кратчайший путь между узлами

```
MATCH path = shortestPath(
  (u:Customer {name: "Иван Петров"})-[*]-(b:Brand {name:
"ASUS"})
)
RETURN path;
```

Пояснение: `shortestPath` находит кратчайший путь между двумя узлами через любые типы связей.

3.5. Обновление данных (Update)

Обновить свойства узла

```
MATCH (p:Product {name: "Galaxy S24"})
SET p.price = 74990, p.onSale = true
RETURN p;
```

Пояснение: `SET` устанавливает новые значения свойств. Можно добавлять новые свойства (`onSale`).

Обновить свойства связи

```
MATCH (u:Customer {name: "Иван Петров"})-[r:REVIEWED]->
(p:Product {name: "Galaxy S24"})
SET r.text = "Отличный смартфон! Пользуюсь уже месяц.", r.edited
= true
RETURN r;
```

Добавить метку к узлу

```
MATCH (p:Product {name: "Galaxy S24"})
SET p:FeaturedProduct
RETURN labels(p);
```

Ожидаемый результат: `["Product", "FeaturedProduct"]` — узел теперь имеет две метки.

Добавить новую связь

```
MATCH (u:Customer {name: "Алексей Козлов"}), (p:Product {name:
"WH-1000XM5"})
CREATE (u)-[:WISHLISTED {addedDate: date("2025-03-10")}]->(p);
```

3.6. Удаление данных (Delete)

Удалить свойство узла

```
MATCH (p:Product {name: "Galaxy S24"})
REMOVE p.onSale
RETURN p;
```

Удалить метку

```
MATCH (p:Product {name: "Galaxy S24"})
REMOVE p:FeaturedProduct
RETURN labels(p);
```

Удалить связь

```
MATCH (u:Customer {name: "Алексей Козлов"})-[r:WISHLISTED]->
(p:Product {name: "WH-1000XM5"})
DELETE r;
```

Важно: удаляется только связь, узлы остаются.

Удалить узел (без связей)

```
CREATE (temp:TempNode {name: "Временный"})
RETURN temp;
```

```
MATCH (temp:TempNode {name: "Временный"})
DELETE temp;
```

Важно: нельзя удалить узел, у которого есть связи, с помощью `DELETE`. Для этого используется `DETACH DELETE`.

Удалить узел со всеми связями

```
MATCH (u:Customer {name: "Алексей Козлов"})
DETACH DELETE u;
```

Пояснение: `DETACH DELETE` сначала удаляет все связи узла, затем сам узел.

Удалить все данные в графе

```
MATCH (n)
DETACH DELETE n;
```

> ***Внимание!*** *Эта команда удалит все узлы и связи. Используйте с осторожностью.*

Часть 4. Elasticsearch

4.1. Теория

Elasticsearch — распределённый поисковый и аналитический движок. Основные понятия:

Понятие	Описание	Аналог в SQL
Index	Индекс (коллекция документов)	Table
Document	JSON-документ	Row
Field	Поле документа	Column
Mapping	Схема индекса (типы полей)	Schema
Analyzer	Анализатор текста (токенизация, нормализация)	—
Query DSL	Язык запросов (JSON)	SQL

Особенности:

- Основан на Apache Lucene.
- Все запросы выполняются через REST API (HTTP).
- Оптимизирован для полнотекстового поиска.
- Поддерживает анализаторы для разных языков.

4.2. Проверка подключения

```
curl -X GET "http://localhost:9200"
```

Ожидаемый результат:

```
{
  "name": "...",
  "cluster_name": "docker-cluster",
  "cluster_uuid": "...",
  "version": {
    "number": "8.x.x",
    ...
  },
  "tagline": "You Know, for Search"
}
```

4.3. Создание структуры данных (Create)

Создание индекса с маппингом

```
curl -X PUT "http://localhost:9200/products" -H "Content-Type:
application/json" -d '
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "analysis": {
      "analyzer": {
        "russian_analyzer": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": ["lowercase", "russian_stop",
"russian_stemmer"]
        }
      },
      "filter": {
        "russian_stop": {
          "type": "stop",
          "stopwords": "_russian_"
        },
```

```
        "russian_stemmer": {
          "type": "stemmer",
          "language": "russian"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "name": {
        "type": "text",
        "analyzer": "russian_analyzer",
        "fields": {
          "keyword": {
            "type": "keyword"
          }
        }
      },
      "description": {
        "type": "text",
        "analyzer": "russian_analyzer"
      },
      "brand": {
        "type": "keyword"
      },
      "category": {
        "type": "keyword"
      },
      "price": {
        "type": "float"
      },
      "rating": {
        "type": "float"
      },
      "inStock": {
        "type": "boolean"
      },
      "tags": {
        "type": "keyword"
      },
      "specs": {
        "type": "object",
        "properties": {
          "ram": { "type": "integer" },
          "storage": { "type": "integer" },
          "display": { "type": "text" }
```

```
        }
      },
      "createdAt": {
        "type": "date"
      }
    }
  }
}'
```

Пояснение:

- `settings` — настройки индекса:
  - `number_of_shards` — количество шардов (1 для разработки).
  - `analysis` — настройка анализаторов. Мы создаём `russian_analyzer` с фильтрами стоп-слов и стеммингом для русского языка.
- `mappings` — схема данных:
  - `text` — полнотекстовое поле (поддерживает поиск по частичному совпадению).
  - `keyword` — точное значение (фильтрация, агрегация, сортировка).
  - `name` имеет мульти-поле `keyword` для точного поиска и сортировки.

Ожидаемый результат:

```
{ "acknowledged": true, "shards_acknowledged": true, "index": "products" }
```

Добавление документов

Добавление одного документа с указанием ID:

```
curl -X PUT "http://localhost:9200/products/_doc/1" -H "Content-Type: application/json" -d '
{
  "name": "Смартфон Samsung Galaxy S24 Ultra",
```

```
  "description": "Флагманский смартфон с титановым корпусом,
камерой 200 МП и встроенным стилусом S Pen. Поддержка
искусственного интеллекта Galaxy AI для обработки фотографий и
перевода в реальном времени.",
  "brand": "Samsung",
  "category": "Смартфоны",
  "price": 109990,
  "rating": 4.8,
  "inStock": true,
  "tags": ["Android", "5G", "NFC", "S Pen", "200MP"],
  "specs": { "ram": 12, "storage": 256, "display": "6.8 дюймов
AMOLED" },
  "createdAt": "2025-01-15"
}'
```

Ожидаемый результат:

```
{
  "_index": "products",
  "_id": "1",
  "_version": 1,
  "result": "created",
  ...
}
```

Добавление нескольких документов одним запросом (Bulk API):

```
curl -X POST "http://localhost:9200/products/_bulk" -H "Content-
Type: application/json" -d '
{"index": {"_id": "2"}}
{"name": "Ноутбук ASUS VivoBook 15", "description": "Тонкий и
лёгкий ноутбук для учёбы и работы. Процессор Intel Core i5 12-го
поколения, экран IPS FullHD, быстрый SSD. Идеален для студентов
и офисных работников.", "brand": "ASUS", "category": "Ноутбуки",
"price": 64990, "rating": 4.6, "inStock": true, "tags":
["Windows", "SSD", "FullHD", "USB-C"], "specs": {"ram": 16,
"storage": 512, "display": "15.6 дюймов IPS"}, "createdAt":
"2025-02-01"}
{"index": {"_id": "3"}}
{"name": "Наушники Sony WH-1000XM5", "description":
"Беспроводные накладные наушники с лучшим в классе активным
шумоподавлением. Поддержка Hi-Res Audio, кодека LDAC, до 30
```

```
часов автономной работы. Комфортная посадка для длительного
прослушивания.", "brand": "Sony", "category": "Аудио", "price":
29990, "rating": 4.9, "inStock": true, "tags": ["Bluetooth",
"ANC", "Hi-Res", "LDAC"], "specs": {"ram": 0, "storage": 0,
"display": "нет"}, "createdAt": "2025-01-20"}
{"index": {"_id": "4"}}
{"name": "Телевизор LG OLED55C3", "description": "55-дюймовый
OLED-телевизор с процессором a9 Gen6 AI. Поддержка Dolby Vision,
Dolby Atmos, 4K 120Гц. Идеален для фильмов и игр благодаря
минимальному времени отклика.", "brand": "LG", "category":
"Телевизоры", "price": 119990, "rating": 4.7, "inStock": true,
"tags": ["OLED", "4K", "SmartTV", "HDR", "120Hz"], "specs":
{"ram": 0, "storage": 0, "display": "55 дюймов OLED"},
"createdAt": "2025-01-10"}
{"index": {"_id": "5"}}
{"name": "Планшет Apple iPad Air M1", "description": "Планшет с
чипом M1, дисплеем Liquid Retina 10.9 дюйма. Поддержка Apple
Pencil 2, Magic Keyboard. Отлично подходит для рисования,
заметок и потребления контента.", "brand": "Apple", "category":
"Планшеты", "price": 59990, "rating": 4.8, "inStock": false,
"tags": ["iPadOS", "M1", "USB-C", "Apple Pencil"], "specs":
{"ram": 8, "storage": 64, "display": "10.9 дюймов Liquid
Retina"}, "createdAt": "2025-02-10"}
'
```

Ожидаемый результат:

```
{
  "took": ...,
  "errors": false,
  "items": [...]
}
```

4.4. Чтение данных (Read)

Получить документ по ID

```
curl -X GET "http://localhost:9200/products/_doc/1"
```

Ожидаемый результат:

```
{
  "_index": "products",
  "_id": "1",
  "_source": {
    "name": "Смартфон Samsung Galaxy S24 Ultra",
    ...
  }
}
```

Получить все документы

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": { "match_all": {} }
}'
```

Полнотекстовый поиск

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "match": {
      "description": "шумоподавление наушники"
    }
  }
}'
```

Пояснение: `match` — полнотекстовый поиск. Elasticsearch анализирует текст запроса тем же анализатором, что и поле `description`, и ищет совпадения. Результаты ранжируются по релевантности (`_score`).

Поиск по фразе

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "match_phrase": {
      "description": "искусственного интеллекта"
    }
  }
}'
```

Пояснение: `match_phrase` — ищет точную фразу (слова должны идти подряд в указанном порядке).

Поиск с фильтрацией (bool query)

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "bool": {
      "must": [
        { "match": { "description": "экран" } }
      ],
      "filter": [
        { "term": { "inStock": true } },
        { "range": { "price": { "gte": 30000, "lte": 100000 } }
}
      ]
    }
  }
}'
```

Пояснение:

- `bool` — составной запрос, комбинирующий несколько условий:
  - `must` — обязательное условие, влияет на `_score`.
  - `filter` — обязательное условие, но НЕ влияет на `_score` (быстрее, кэшируется).
  - `should` — желательное условие (повышает `_score`).

    - `must_not` — исключающее условие.
- `term` — точное совпадение (для `keyword` и `boolean` полей).
- `range` — диапазон значений.

Поиск по нескольким полям

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "multi_match": {
      "query": "студент учёба",
      "fields": ["name", "description"],
      "type": "best_fields"
    }
  }
}'
```

Поиск по тегам (точное совпадение)

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "terms": { "tags": ["5G", "NFC"] }
  }
}'
```

Поиск с подсветкой (highlight)

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": {
    "match": { "description": "камера фотография" }
  },
```

```
  "highlight": {
    "fields": {
      "description": {}
    }
  }
}'
```

Пояснение: в ответе найденные слова будут обёрнуты в теги `<em>...</em>`.

Агрегации

Средняя цена по категориям:

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "size": 0,
  "aggs": {
    "categories": {
      "terms": { "field": "category" },
      "aggs": {
        "avg_price": { "avg": { "field": "price" } },
        "max_price": { "max": { "field": "price" } }
      }
    }
  }
}'
```

Пояснение:

- `"size": 0` — не возвращать сами документы, только агрегации.
- `terms` — группировка по значению поля.
- Вложенные `aggs` — агрегации внутри каждой группы.

Сортировка и пагинация

```
curl -X GET "http://localhost:9200/products/_search" -H
"Content-Type: application/json" -d '
{
  "query": { "match_all": {} },
```

```
  "sort": [
    { "price": { "order": "asc" } }
  ],
  "from": 0,
  "size": 3,
  "_source": ["name", "price", "brand"]
}'
```

Пояснение:

- `sort` — сортировка.
- `from` и `size` — пагинация (пропустить `from` документов, вернуть `size`).
- `_source` — какие поля вернуть (проекция).

4.5. Обновление данных (Update)

Частичное обновление документа

```
curl -X POST "http://localhost:9200/products/_update/1" -H
"Content-Type: application/json" -d '
{
  "doc": {
    "price": 99990,
    "rating": 4.9,
    "tags": ["Android", "5G", "NFC", "S Pen", "200MP", "Galaxy
AI"]
  }
}'
```

Пояснение: `doc` — объект с полями, которые нужно обновить. Остальные поля не изменяются.

Ожидаемый результат:

```
{
  "_index": "products",
  "_id": "1",
```

```
  "_version": 2,
  "result": "updated"
}
```

Обновление с использованием скрипта

```
curl -X POST "http://localhost:9200/products/_update/1" -H
"Content-Type: application/json" -d '
{
  "script": {
    "source": "ctx._source.price -= params.discount",
    "params": {
      "discount": 10000
    }
  }
}'
```

Пояснение: `script` позволяет выполнить произвольную логику. `ctx._source` — текущий документ. Цена уменьшится на 10000.

Обновление по запросу (update by query)

```
curl -X POST "http://localhost:9200/products/_update_by_query" -
H "Content-Type: application/json" -d '
{
  "query": {
    "term": { "brand": "Samsung" }
  },
  "script": {
    "source": "ctx._source.inStock = params.status",
    "params": {
      "status": true
    }
  }
}'
```

Пояснение: обновляет все документы, соответствующие запросу.

4.6. Удаление данных (Delete)

Удалить документ по ID

```
curl -X DELETE "http://localhost:9200/products/_doc/5"
```

Ожидаемый результат:

```
{
  "_index": "products",
  "_id": "5",
  "result": "deleted"
}
```

Удалить документы по запросу

```
curl -X POST "http://localhost:9200/products/_delete_by_query" -
H "Content-Type: application/json" -d '
{
  "query": {
    "term": { "inStock": false }
  }
}'
```

Удалить индекс (хранилище)

```
curl -X DELETE "http://localhost:9200/products"
```

Ожидаемый результат:

```
{ "acknowledged": true }
```

Просмотр маппинга индекса

```
curl -X GET "http://localhost:9200/products/_mapping"
```

Просмотр настроек индекса

```
curl -X GET "http://localhost:9200/products/_settings"
```

Часть 5. PostgreSQL

5.1. Теория

PostgreSQL — объектно-реляционная СУБД. Основные понятия:

Понятие	Описание
Database	База данных
Schema	Пространство имён (по умолчанию `public`)
Table	Таблица
Row	Строка (запись)
Column	Столбец (поле)
Index	Индекс для ускорения поиска
Constraint	Ограничение (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK)
Transaction	Транзакция (набор операций, выполняемых атомарно)

Расширенные возможности PostgreSQL:

- **JSONB** — эффективное хранение JSON-данных с возможностью индексирования.

- **Массивы** — нативная поддержка массивов в столбцах.
- **hstore** — хранилище пар ключ-значение в одном столбце.

5.2. Подключение

```
docker exec -it postgres psql -U postgres
```

5.3. Создание структуры данных (Create)

Создание базы данных

```
CREATE DATABASE electronics_shop;
\c electronics_shop
```

Пояснение: `\c` — команда psql для переключения на другую базу данных.

Подключение расширения hstore

```
CREATE EXTENSION IF NOT EXISTS hstore;
```

Создание таблиц

```
-- Таблица категорий
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица брендов
CREATE TABLE brands (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
```

```
    country VARCHAR(100),
    founded_year INTEGER,
    website VARCHAR(255)
);

-- Таблица товаров с JSONB, массивами и hstore
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    brand_id INTEGER REFERENCES brands(id) ON DELETE SET NULL,
    category_id INTEGER REFERENCES categories(id) ON DELETE SET
NULL,
    price NUMERIC(10, 2) NOT NULL CHECK (price > 0),
    in_stock BOOLEAN DEFAULT true,
    rating NUMERIC(2, 1) DEFAULT 0.0 CHECK (rating >= 0 AND
rating <= 5),
    tags TEXT[] DEFAULT '{}',
    specs JSONB DEFAULT '{}',
    attributes hstore DEFAULT '',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица покупателей
CREATE TABLE customers (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    age INTEGER CHECK (age > 0 AND age < 150),
    address JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица заказов
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(id) ON
DELETE CASCADE,
    total_amount NUMERIC(12, 2) NOT NULL,
    status VARCHAR(50) DEFAULT 'new',
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Таблица позиций заказа (связь многие-ко-многим)
CREATE TABLE order_items (
```

```
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE
CASCADE,
    product_id INTEGER NOT NULL REFERENCES products(id) ON
DELETE CASCADE,
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10, 2) NOT NULL
);

-- Таблица отзывов
CREATE TABLE reviews (
    id SERIAL PRIMARY KEY,
    product_id INTEGER NOT NULL REFERENCES products(id) ON
DELETE CASCADE,
    customer_id INTEGER NOT NULL REFERENCES customers(id) ON
DELETE CASCADE,
    rating INTEGER NOT NULL CHECK (rating >= 1 AND rating <= 5),
    text TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(product_id, customer_id)
);
```

Пояснение к структуре:

- `SERIAL` — автоинкрементный целочисленный тип.
- `PRIMARY KEY` — первичный ключ (уникальный, NOT NULL).
- `REFERENCES` — внешний ключ (ссылка на другую таблицу).
- `ON DELETE CASCADE` — при удалении родительской записи автоматически удаляются связанные.
- `ON DELETE SET NULL` — при удалении родительской записи поле устанавливается в NULL.
- `CHECK` — ограничение на допустимые значения.
- `TEXT[]` — массив строк.
- `JSONB` — бинарный JSON (поддерживает индексы и операторы).
- `hstore` — хранилище пар ключ-значение.

Создание индексов

```sql
-- Индекс для быстрого поиска по категории
CREATE INDEX idx_products_category ON products(category_id);

-- Индекс для быстрого поиска по бренду
CREATE INDEX idx_products_brand ON products(brand_id);

-- GIN-индекс для полнотекстового поиска по JSONB
CREATE INDEX idx_products_specs ON products USING GIN(specs);

-- GIN-индекс для поиска по массивам тегов
CREATE INDEX idx_products_tags ON products USING GIN(tags);

-- GIN-индекс для hstore
CREATE INDEX idx_products_attributes ON products USING
GIN(attributes);

-- Индекс для поиска по email покупателя
CREATE INDEX idx_customers_email ON customers(email);
```

Пояснение:

- `B-tree` (по умолчанию) — для обычных столбцов, поддерживает сравнения (`=` , `<` , `>` , `BETWEEN`).
- `GIN` (Generalized Inverted Index) — для составных типов (JSONB, массивы, hstore, полнотекстовый поиск).

5.4. Добавление данных (Create — INSERT)

```sql
-- Категории
INSERT INTO categories (name, description) VALUES
    ('Смартфоны', 'Мобильные телефоны и смартфоны'),
    ('Ноутбуки', 'Портативные компьютеры'),
    ('Аудио', 'Наушники и аудиотехника'),
    ('Телевизоры', 'Телевизоры и мониторы'),
    ('Планшеты', 'Планшетные компьютеры');

-- Бренды
INSERT INTO brands (name, country, founded_year, website) VALUES
    ('Samsung', 'Южная Корея', 1938, 'https://samsung.com'),
    ('ASUS', 'Тайвань', 1989, 'https://asus.com'),
```

```
    ('Sony', 'Япония', 1946, 'https://sony.com'),
    ('LG', 'Южная Корея', 1958, 'https://lg.com'),
    ('Apple', 'США', 1976, 'https://apple.com');

-- Товары с массивами, JSONB и hstore
INSERT INTO products (name, brand_id, category_id, price,
in_stock, rating, tags, specs, attributes) VALUES
(
    'Смартфон Samsung Galaxy S24 Ultra',
    1, 1, 109990, true, 4.8,
    ARRAY['Android', '5G', 'NFC', 'S Pen', '200MP'],
    '{"display": "6.8 дюймов AMOLED", "processor": "Snapdragon 8
Gen 3", "ram": 12, "storage": 256, "battery": 5000, "camera":
{"main": "200MP", "ultra_wide": "12MP", "telephoto": "50MP"}}',
    'color => "Titanium Gray", weight => "232г", os_version =>
"Android 14"'
),
(
    'Ноутбук ASUS VivoBook 15',
    2, 2, 64990, true, 4.6,
    ARRAY['Windows', 'SSD', 'FullHD', 'USB-C'],
    '{"display": "15.6 дюймов IPS FullHD", "processor": "Intel
Core i5-1235U", "ram": 16, "storage": 512, "gpu": "Intel Iris
Xe", "ports": ["USB-A", "USB-C", "HDMI", "3.5mm"]}',
    'color => "Silver", weight => "1.7кг", os_version =>
"Windows 11"'
),
(
    'Наушники Sony WH-1000XM5',
    3, 3, 29990, true, 4.9,
    ARRAY['Bluetooth', 'ANC', 'Hi-Res', 'LDAC'],
    '{"type": "Накладные", "connection": "Bluetooth 5.2",
"noise_cancellation": true, "battery_life_hours": 30, "codecs":
["SBC", "AAC", "LDAC"]}',
    'color => "Black", weight => "250г", driver_size => "30мм"'
),
(
    'Телевизор LG OLED55C3',
    4, 4, 119990, true, 4.7,
    ARRAY['OLED', '4K', 'SmartTV', 'HDR', '120Hz'],
    '{"display": "55 дюймов OLED 4K", "processor": "a9 Gen6 AI",
"smart_tv": true, "hdr": ["Dolby Vision", "HDR10", "HLG"],
"refresh_rate": 120}',
    'color => "Black", weight => "14.2кг", wall_mount =>
"300x300mm"'
),
```

```
(
    'Планшет Apple iPad Air M1',
    5, 5, 59990, false, 4.8,
    ARRAY['iPadOS', 'M1', 'USB-C', 'Apple Pencil'],
    '{"display": "10.9 дюймов Liquid Retina", "processor":
"Apple M1", "ram": 8, "storage": 64, "biometric": "Touch ID"}',
    'color => "Space Gray", weight => "461г", os_version =>
"iPadOS 17"'
);

-- Покупатели с JSONB-адресом
INSERT INTO customers (name, email, age, address) VALUES
(
    'Иван Петров', 'ivan@example.com', 28,
    '{"city": "Москва", "street": "ул. Тверская, д. 10", "zip":
"125009"}'
),
(
    'Мария Сидорова', 'maria@example.com', 34,
    '{"city": "Санкт-Петербург", "street": "Невский пр., д. 50",
"zip": "191025"}'
),
(
    'Алексей Козлов', 'alexey@example.com', 22,
    '{"city": "Казань", "street": "ул. Баумана, д. 15", "zip":
"420111"}'
);

-- Заказы
INSERT INTO orders (customer_id, total_amount, status) VALUES
    (1, 139980, 'completed'),
    (2, 64990, 'shipped'),
    (3, 219980, 'processing');

-- Позиции заказов
INSERT INTO order_items (order_id, product_id, quantity,
unit_price) VALUES
    (1, 1, 1, 109990),
    (1, 3, 1, 29990),
    (2, 2, 1, 64990),
    (3, 1, 2, 109990);

-- Отзывы
INSERT INTO reviews (product_id, customer_id, rating, text)
VALUES
    (1, 1, 5, 'Отличный смартфон! Камера просто потрясающая.'),
```

```
    (1, 3, 4, 'Хороший, но дороговат.'),
    (2, 2, 4, 'Хороший ноутбук для учёбы и работы.'),
    (3, 1, 5, 'Лучшее шумоподавление, что я пробовал.');
```

5.5. Чтение данных (Read — SELECT)

Простая выборка

```
SELECT * FROM products;
```

Выборка с условием и выбором полей

```
SELECT name, price, rating
FROM products
WHERE in_stock = true AND price < 100000
ORDER BY price ASC;
```

Ожидаемый результат:

name	price	rating
Наушники Sony WH-1000XM5	29990.00	4.9
Ноутбук ASUS VivoBook 15	64990.00	4.6

JOIN — соединение таблиц

```
SELECT
    p.name AS product,
    b.name AS brand,
    c.name AS category,
    p.price,
    p.rating
FROM products p
JOIN brands b ON p.brand_id = b.id
```

```
JOIN categories c ON p.category_id = c.id
ORDER BY p.price DESC;
```

Пояснение: `JOIN` (или `INNER JOIN`) соединяет строки из разных таблиц по условию. Псевдонимы (`p`, `b`, `c`) сокращают запись.

Работа с JSONB

Извлечение значения из JSON:

```
-- Оператор -> возвращает JSON-объект
SELECT name, specs->'processor' AS processor FROM products;

-- Оператор ->> возвращает значение как текст
SELECT name, specs->>'processor' AS processor FROM products;

-- Доступ к вложенным полям
SELECT name, specs->'camera'->>'main' AS main_camera
FROM products
WHERE specs->'camera' IS NOT NULL;
```

Поиск по JSONB:

```
-- Товары с RAM >= 8
SELECT name, price
FROM products
WHERE (specs->>'ram')::integer >= 8;

-- Оператор @> — содержит ли JSONB указанный подобъект
SELECT name
FROM products
WHERE specs @> '{"smart_tv": true}';

-- Поиск по вложенному массиву в JSONB
SELECT name
FROM products
WHERE specs->'codecs' ? 'LDAC';
```

Основные операторы JSONB:

Оператор	Описание
`->`	Получить элемент по ключу как JSON
`->>`	Получить элемент по ключу как TEXT
`@>`	Левый JSON содержит правый
`<@`	Левый JSON содержится в правом
`?`	Ключ/элемент существует
`?|`	Любой из ключей существует
`?&`	Все ключи существуют

Работа с массивами

```sql
-- Товары с тегом '5G'
SELECT name, tags
FROM products
WHERE '5G' = ANY(tags);

-- Товары, содержащие все указанные теги
SELECT name, tags
FROM products
WHERE tags @> ARRAY['USB-C', 'SSD'];

-- Товары, содержащие хотя бы один из тегов
SELECT name, tags
FROM products
WHERE tags && ARRAY['OLED', 'AMOLED'];

-- Длина массива тегов
SELECT name, array_length(tags, 1) AS tag_count
FROM products
ORDER BY tag_count DESC;
```

Основные операторы массивов:

Оператор	Описание
`ANY(array)`	Элемент содержится в массиве

`@>`	Массив содержит все элементы другого
`<@`	Массив содержится в другом
`&&`	Массивы пересекаются (есть общие элементы)
`array_length(arr, dim)`	Длина массива
`unnest(arr)`	Развернуть массив в строки

Работа с hstore

```
-- Получить значение по ключу
SELECT name, attributes->'color' AS color
FROM products;

-- Фильтрация по hstore
SELECT name, attributes
FROM products
WHERE attributes->'color' = 'Black';

-- Проверка наличия ключа
SELECT name
FROM products
WHERE attributes ? 'wall_mount';

-- Получить все ключи
SELECT name, akeys(attributes) AS keys
FROM products;
```

Агрегатные функции

```
SELECT
    c.name AS category,
    COUNT(p.id) AS product_count,
    ROUND(AVG(p.price), 2) AS avg_price,
    MIN(p.price) AS min_price,
    MAX(p.price) AS max_price
FROM products p
JOIN categories c ON p.category_id = c.id
```

```
GROUP BY c.name
ORDER BY avg_price DESC;
```

Подзапросы

```
-- Товары дороже средней цены
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);

-- Покупатели, сделавшие заказы
SELECT name, email
FROM customers
WHERE id IN (SELECT DISTINCT customer_id FROM orders);
```

Сложный запрос с JOIN и агрегацией

```
-- Покупатели и сумма их заказов
SELECT
    cu.name,
    cu.email,
    COUNT(o.id) AS order_count,
    SUM(o.total_amount) AS total_spent
FROM customers cu
JOIN orders o ON cu.id = o.customer_id
GROUP BY cu.id, cu.name, cu.email
ORDER BY total_spent DESC;
```

Оконные функции

```
-- Ранжирование товаров по цене внутри категории
SELECT
    p.name,
    c.name AS category,
    p.price,
    RANK() OVER (PARTITION BY c.name ORDER BY p.price DESC) AS
price_rank
```

```
FROM products p
JOIN categories c ON p.category_id = c.id;
```

5.6. Обновление данных (Update — UPDATE)

Простое обновление

```
UPDATE products
SET price = 99990, updated_at = CURRENT_TIMESTAMP
WHERE name = 'Смартфон Samsung Galaxy S24 Ultra';
```

Ожидаемый результат: `UPDATE 1`.

Обновление с JOIN

```
UPDATE products
SET price = price * 0.9, updated_at = CURRENT_TIMESTAMP
FROM brands
WHERE products.brand_id = brands.id AND brands.name = 'Samsung';
```

Пояснение: скидка 10% на все товары Samsung.

Обновление JSONB

```
-- Добавить/изменить поле в JSONB
UPDATE products
SET specs = specs || '{"5g": true}'
WHERE name LIKE '%Galaxy%';

-- Удалить поле из JSONB
UPDATE products
SET specs = specs - 'battery'
WHERE name LIKE '%Galaxy%';

-- Обновить вложенное поле
UPDATE products
```

```
SET specs = jsonb_set(specs, '{ram}', '16')
WHERE name = 'Ноутбук ASUS VivoBook 15';
```

Основные функции для JSONB:

Функция/Оператор	Описание
`||`	Объединить два JSONB
`-`	Удалить ключ
`jsonb_set(target, path, value)`	Установить значение по пути
`jsonb_insert(target, path, value)`	Вставить значение

Обновление массива

```
-- Добавить элемент в массив
UPDATE products
SET tags = array_append(tags, 'Bestseller')
WHERE name = 'Наушники Sony WH-1000XM5';

-- Удалить элемент из массива
UPDATE products
SET tags = array_remove(tags, 'Bestseller')
WHERE name = 'Наушники Sony WH-1000XM5';

-- Заменить массив целиком
UPDATE products
SET tags = ARRAY['Android', '5G', 'NFC', 'S Pen', '200MP',
'Galaxy AI']
WHERE name LIKE '%Galaxy S24%';
```

Обновление hstore

```
-- Добавить/изменить пару ключ-значение
UPDATE products
SET attributes = attributes || 'warranty => "2 года"'::hstore
WHERE name LIKE '%Galaxy%';

-- Удалить ключ
```

```sql
UPDATE products
SET attributes = delete(attributes, 'warranty')
WHERE name LIKE '%Galaxy%';
```

Транзакции

```sql
BEGIN;

-- Создаём заказ
INSERT INTO orders (customer_id, total_amount, status) VALUES
(1, 29990, 'new')
RETURNING id;

-- Допустим, вернулся id = 4
INSERT INTO order_items (order_id, product_id, quantity,
unit_price) VALUES (4, 3, 1, 29990);

-- Уменьшаем остаток (если бы было поле stock)
-- UPDATE products SET stock = stock - 1 WHERE id = 3;

COMMIT;
```

Пояснение:

- `BEGIN` — начало транзакции.
- `COMMIT` — фиксация транзакции (все изменения сохраняются).
- `ROLLBACK` — откат транзакции (все изменения отменяются).
- `RETURNING id` — возвращает значение после INSERT.

Пример отката:

```sql
BEGIN;

UPDATE products SET price = 0 WHERE id = 1;

-- Ой, ошибка! Откатываем
ROLLBACK;
```

```
-- Цена не изменилась
SELECT price FROM products WHERE id = 1;
```

5.7. Удаление данных (Delete — DELETE)

Удаление конкретной записи

```
DELETE FROM reviews WHERE id = 1;
```

Ожидаемый результат: `DELETE 1`.

Удаление с условием

```
DELETE FROM reviews WHERE rating < 3;
```

Удаление с подзапросом

```
DELETE FROM order_items
WHERE order_id IN (
    SELECT id FROM orders WHERE status = 'cancelled'
);
```

Очистка таблицы (удаление всех строк)

```
-- DELETE удаляет построчно (медленнее, пишет в WAL)
DELETE FROM reviews;

-- TRUNCATE удаляет мгновенно (сбрасывает таблицу)
TRUNCATE TABLE reviews RESTART IDENTITY CASCADE;
```

Пояснение:

- `TRUNCATE` быстрее `DELETE` для больших таблиц, но не вызывает триггеры.
- `RESTART IDENTITY` — сбрасывает счётчик SERIAL.
- `CASCADE` — также очищает таблицы, ссылающиеся на эту через FOREIGN KEY.

Удаление таблицы

```
DROP TABLE IF EXISTS reviews;
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS brands;
```

Пояснение: `IF EXISTS` предотвращает ошибку, если таблица уже удалена. Порядок важен — сначала удаляются зависимые таблицы.

Альтернативно, каскадное удаление:

```
DROP TABLE IF EXISTS products CASCADE;
```

Пояснение: `CASCADE` автоматически удалит все зависимые объекты (внешние ключи, представления).

Удаление базы данных

```
-- Сначала отключиться от базы
\c postgres

-- Затем удалить
DROP DATABASE IF EXISTS electronics_shop;
```

Сводная таблица CRUD-операций

Операция	Redis	MongoDB	Neo4j (Cypher)	
Создать хранилище	Автоматически	`use db` + автоматически	Автоматически	
Добавить	`SET`, `HSET`, `LPUSH`, `SADD`, `ZADD`	`insertOne()`, `insertMany()`	`CREATE`	
Прочитать	`GET`, `HGETALL`, `LRANGE`, `SMEMBERS`, `ZRANGE`	`find()`, `findOne()`	`MATCH ... RETURN`	
Обновить	`SET`, `HSET`, `LSET`, `ZADD`	`updateOne()`, `updateMany()`	`MATCH ... SET`	
Удалить данные	`DEL`, `HDEL`, `LREM`, `SREM`, `ZREM`	`deleteOne()`, `deleteMany()`	`DELETE`, `DETACH DELETE`	
Удалить хранилище	`FLUSHDB`	`db.collection.drop()`, `db.dropDatabase()`	`MATCH (n) DETACH DELETE n`	

Контрольные вопросы

1. Что означает аббревиатура CRUD? Перечислите операции и их SQL-аналоги.
2. **Redis:**
    - Чем отличаются типы данных String, Hash, List, Set и Sorted Set? Приведите примеры задач, для которых подходит каждый тип.

- Что произойдёт, если выполнить `SET` для уже существующего ключа?
- Чем отличается `LPUSH` от `RPUSH`? В каких случаях используется каждая команда?
- Что такое TTL и для чего он используется?
- Чем отличается `DEL` от `FLUSHDB`?

3. **MongoDB:**

- Что такое вложенный документ (embedded document)? Приведите пример.
- Чем отличается `updateOne()` от `updateMany()`?
- Для чего используется оператор `$push`? Чем он отличается от `$set`?
- Что такое агрегация? Опишите конвейер агрегации (aggregation pipeline).
- Что делает параметр `upsert: true`?

4. **Neo4j:**

- Что такое узел, связь и свойство в графовой модели?
- Чем отличается `DELETE` от `DETACH DELETE`? Почему важно это различие?
- Что делает команда `MERGE` и чем она отличается от `CREATE`?
- Приведите пример запроса на Cypher, который находит все товары определённого бренда.
- Как представить связь «многие-ко-многим» в графовой модели?

5. **Elasticsearch:**

- Что такое маппинг (mapping)? Зачем он нужен?
- Чем отличаются типы полей `text` и `keyword`?
- Что такое анализатор (analyzer)? Из каких компонентов он состоит?
- Чем отличается `match` от `match_phrase` в поисковых запросах?
- Что такое `bool` query? Перечислите его составные части (`must`, `filter`, `should`, `must_not`) и объясните разницу.

6. **PostgreSQL:**
    - Что такое внешний ключ (FOREIGN KEY)? Для чего он нужен?
    - Чем отличается `DELETE` от `TRUNCATE`?
    - Что такое транзакция? Объясните команды `BEGIN`, `COMMIT`, `ROLLBACK`.
    - Приведите пример запроса с использованием оператора JSONB `@>`.
    - Чем GIN-индекс отличается от B-tree? Для каких типов данных используется каждый?
    - Что такое hstore и чем он отличается от JSONB?
7. **Общие вопросы:**
    - Сравните модели данных всех пяти СУБД. Какая модель лучше подходит для: а) кэширования сессий; б) каталога товаров; в) социальной сети; г) поисковой системы; д) финансового учёта?
    - Что такое схема данных (schema)? В каких из рассмотренных СУБД схема является строгой, а в каких — гибкой?
    - Какие преимущества и недостатки даёт хранение данных в оперативной памяти (на примере Redis)?

Список источников

1. Документация Redis — https://redis.io/docs
2. Документация MongoDB — https://www.mongodb.com/docs/manual/
3. Документация Neo4j (Cypher) — https://neo4j.com/docs/cypher-manual/
4. Документация Elasticsearch — https://www.elastic.co/guide/en/elasticsearch/reference/current/
5. Документация PostgreSQL — https://www.postgresql.org/docs/current/